

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении лабораторной работы №1
по дисциплине «Математическая логика и теория формальных языков»
«Лексический анализатор»
Вариант №9

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Востров А. В.

«_____» _____ 2026г.

Содержание

Введение	3
1 Математическое описание	4
1.1 Структура транслятора	4
1.2 Лексический анализ	4
1.3 Формальная модель лексического анализатора	5
1.4 Синтаксические диаграммы	6
2 Особенности реализации	9
2.1 Структуры данных	9
2.2 Реализация класса лексического анализатора	9
2.2.1 Основные методы	10
2.2.2 Регулярные выражения	13
3 Результаты работы программы	14
Заключение	17
Список использованной литературы	18

Введение

В лабораторной работе необходимо написать программу, которая выполняет лексический анализ входного текста в соответствии с вариантом и порождает таблицу лексем с указанием их типов и значений. Также должны реализовываться следующие задачи:

- Программа должна выдавать сообщения о наличии во входном тексте ошибок, которые могут быть обнаружены на этапе лексического анализа.
- Программа должна допускать наличие комментариев неограниченной длины во входном файле. Комментарий - это последовательность символов, которая с обеих сторон ограничена решеткой (`# comment #`).

Правила для анализа (9 вариант): входной язык содержит только латиницу, арифметические выражения, разделенные символом `;` (точка с запятой). Арифметические выражения состоят из идентификаторов, римских чисел, знака присваивания (`:=`), знаков операций `+`, `-`, `*`, `/` и круглых скобок. Римскими считать числа, записанные большими буквами `X`, `V` и `I`. Длина идентификатора и строковых констант ограничена 15 символами.

1 Математическое описание

1.1 Структура транслятора

Транслятор преобразует исходный текст L на каком-либо языке во внутреннюю структуру данных с сохранением смысла исходного текста. В соответствии с идеей синтаксически-ориентированной трансляции, процесс трансляции в информатике связывается с двумя основными этапами:

1. Распознаватель строит структуру входного текста на основе порождающей грамматики входного языка.
2. Генератор использует построенную структуру для создания выходных данных, отражающих семантику входной цепочки.

На рисунке 1 приведена схема транслятора.



Рис. 1. Схема транслятора

Во многих трансляторах процессы распознавания и генерации могут быть разделены не так явно, однако метод синтаксически-ориентированной трансляции всегда основывается на построении структуры входных данных.

1.2 Лексический анализ

Лексический анализ — первая фаза трансляции, основная задача которой состоит в чтении входных символов исходной программы и их группировании в лексемы.

Лексемы — минимальные единицы языка, имеющие смысл. Это может быть не один символ, а их последовательность. Например, в языках программирования лексемами являются имена, служебные слова, константы и операторы.

Цели лексического анализа:

- Определение класса лексемы — категории элементов с общими свойствами (идентификатор, целое число, строка символов, оператор и др.)

- Определение значения лексемы (подстроки символов входной цепочки) в соответствии с распознанным классом.
- Преобразование значений лексем некоторых классов (например, чисел) во внутреннее представление.

Лексический анализатор обрабатывает входную цепочку, а на его вход подаются символы, сгруппированные по категориям. Поэтому перед лексическим анализом осуществляется дополнительная обработка, сопоставляющая с каждым символом его класс, что позволяет сканеру манипулировать единым понятием для целой группы символов.

Устройство, осуществляющее сопоставление класса с каждым отдельным символом, называется транслитератором.

1.3 Формальная модель лексического анализатора

Лексический анализатор - это конечный автомат, который преобразует входную строку символов в последовательность токенов. Формально его можно описать следующим образом:

Пусть заданы:

- Σ — входной алфавит (множество допустимых символов), где $\Sigma = \{A, \dots, Z, a, \dots, z, ;, :=, +, -, *, /, (,)\}$.
В алфавит входят $\Sigma = \text{Latin} \cup \text{Ariphm} \cup \text{Other}$:
 - **Latin** — латинские буквы обоих регистров: $\{A, \dots, Z, a, \dots, z\}$,
 - **Ariphm** — знаки арифметических операций: $\{+, -, *, /\}$,
 - **Other** — специальные символы: $\{;, :=, (,)\}$.
- T — множество типов токенов, где $T = \{\text{ASSIGN}, \text{SEP}, \text{ARITHM}, \text{L_BRACKET}, \text{R_BRACKET}, \text{ID}, \text{ROMAN}, \text{ERROR}\}$:
 - **ASSIGN** — тип токена, обозначающий знак присваивания ($:=$).
 - **SEP** — тип токена, обозначающий разделитель ($;$).
 - **ARITHM** — тип токена, обозначающий арифметические операции ($+$, $-$, $*$, $/$).
 - **L_BRACKET** — тип токена, обозначающий открывающую круглую скобку.
 - **R_BRACKET** — тип токена, обозначающий закрывающую круглую скобку.
 - **ID** — тип токена, обозначающий идентификатор.
 - **ROMAN** — тип токена, обозначающий римское число.

- **ERROR** — тип токена, обозначающий ошибку. Токены с этим типом не попадают в таблицу лексем.
- D — множество допустимых значений токенов. Оно разбито на непересекающиеся классы, которые соответствуют типам токенов:
 $D = D_{\text{ID}} \cup D_{\text{ROMAN}} \cup D_{\text{OPERATORS}} \cup D_{\text{ASSIGN}}$.
 - Для типа **ID**: $D_{\text{ID}} = \{s \mid s \in L_f \cdot L^{n-1}, 1 \leq n \leq 15\}$, где $L = \{A \dots Z, a \dots z\}$, $L_f = L \setminus \{X, V, I\}$, — строка s длины n , которая начинается с любой латинской буквы, кроме X, V, I ;
 - Для типа **ROMAN**: $D_{\text{ROMAN}} = \{s \mid s \in R^k, k \geq 1, R = \{X, V, I\}\}$ — последовательность из заглавных латинских букв X, V, I ;
 - Для типа **OPERATORS**: $D_{\text{OPERATORS}} = \{+, -, *, /, (,), ;\}$;
 - Для типа **ASSIGN**: $D_{\text{ASSIGN}} = \{:=\}$.

Если последовательность символов не входит в D , она распознается как ошибка, которая не обрабатывается лексическим анализатором.

Тогда лексический анализатор реализует отображение

$$F_{\text{lexer}} : \Sigma^* \rightarrow (T \times D)^*, \text{ где}$$

- Σ^* — множество всех возможных строк над алфавитом Σ
- $(T \times D)^*$ — множество последовательностей пар вида (тип токена, значение)

1.4 Синтаксические диаграммы

Синтаксические диаграммы — это направленные графы с одним входом и одним выходом. Вершины графа помечены символами, которые могут встретиться в цепочке языка. Последовательность символов, встреченных на любом пути от входа к выходу синтаксической диаграммы, является цепочкой языка, задаваемого этой синтаксической диаграммой.

Синтаксические диаграммы визуализируют правила формирования лексем.

В данной лабораторной работе лексический анализатор обрабатывает следующие классы лексем:

1. Идентификаторы (последовательность букв латинского алфавита (не может начинаться с последовательности заглавных букв X, V, I), максимальная длина 15 символов)
2. Римские числа (построенные по правилам римского исчисления, содержащие только X, V, I)

3. Оператор присваивания :=
4. Арифметические операторы (знаки +, -, *, /)
5. Левая круглая скобка (
6. Правая круглая скобка)
7. Разделитель выражений ;
8. Комментарий (начинается с # и заканчивается #)

На рисунке 2 представлена диаграмма работы лексического анализатора в данной работе.

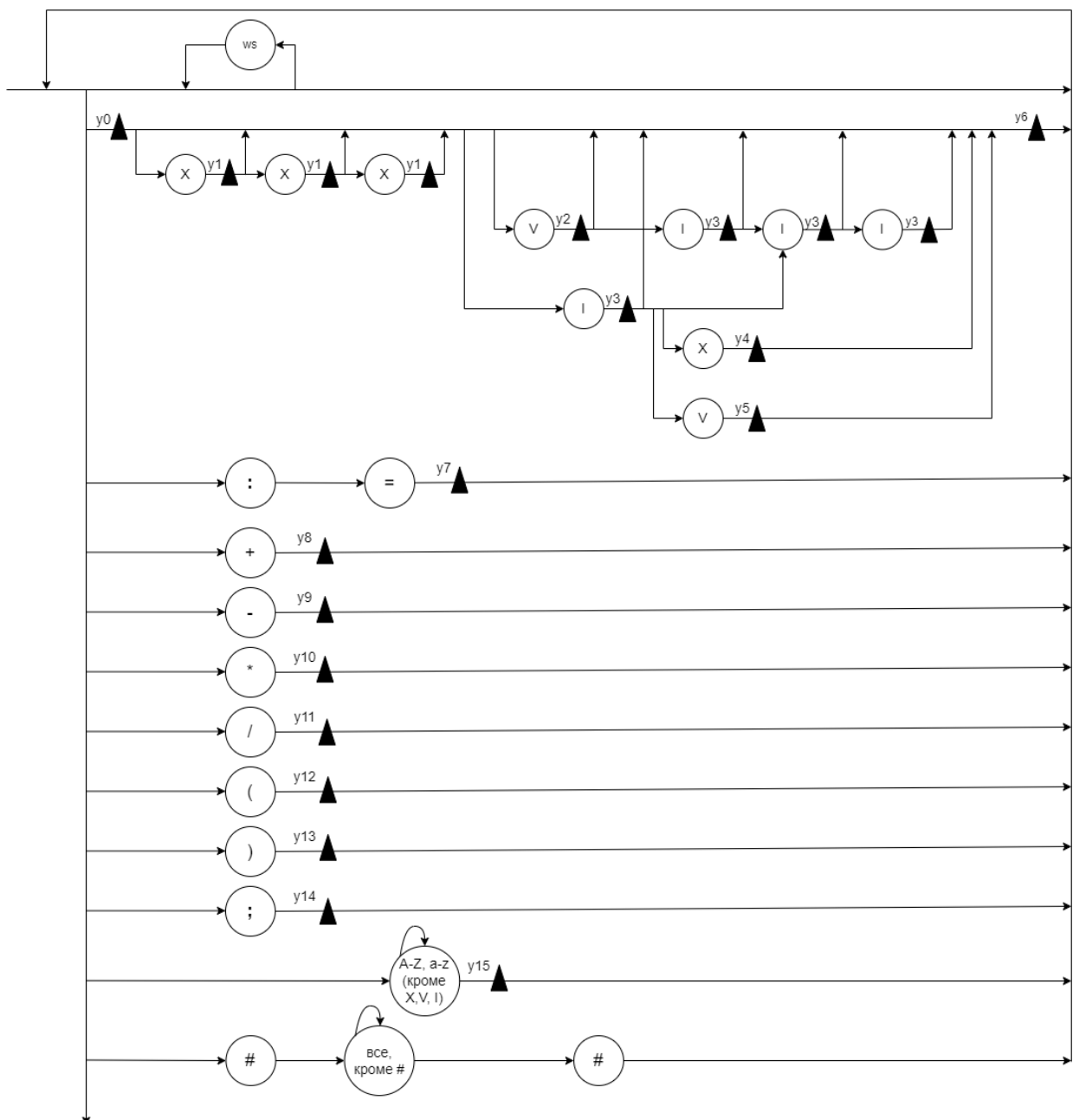


Рис. 2. Синтаксическая диаграмма

Семантические действия:

y0 : число = 0

y1 : число +=10

y2 : число +=5

y3 : число +=1

y4 : число +=8

y5 : число +=3

y6 : записать число, выдать лексему <число, значение>

y7 : выдать лексему :=

y8 : выдать лексему +

y9 : выдать лексему —

y10 : выдать лексему *

y11 : выдать лексему /

y12 : выдать лексему (

y13 : выдать лексему)

y14 : выдать лексему ;

y15 : записать идентификатор, выдать лексему <id, значение>

Обработка синтаксических ошибок на диаграмме не показывается, чтобы не загромождать структуру, но в реализации она присутствует. Распознаются следующие ошибки:

- Некорректные римские числа.
- Недопустимые символы, не принадлежащие входному языку.
- Идентификаторы длиной более 15 символов и/или содержащие недопустимые символы (X,V,I и символы, не принадлежащие входному языку).

2 Особенности реализации

2.1 Структуры данных

Лексический анализатор представлен следующими структурами данных:

1. **errors** - список строк для накопления и отображения ошибок, возникающих при лексическом и синтаксическом анализе строки.
2. **lex_table** - список кортежей вида (тип лексемы, значение) для хранения найденных лексем.
3. **expression** - строка хранения изначального текста.
4. **identifiers** - словарь для хранения уникальных номеров идентификаторов. Ключами словаря являются идентификаторы, а значениями - целые числа.
5. **id_counter** - целое число для присвоения уникального номера идентификаторам.
6. **reg_roman**, **reg_id** - регулярные выражения для определения допустимых римских чисел и идентификаторов.

2.2 Реализация класса лексического анализатора

Класс **Analyzator** представляет структуру, которая обрабатывает лексическим анализатором входящую строку, проверяет ее синтаксис и сохраняет результаты анализа. Код анализатора представлен в приложении А.

Конструктор класса **__init__** инициализирует атрибуты экземпляра.

Вход: ссылка на экземпляр класса **self**

Выход: пустые список ошибок, таблица лексем, таблица идентификаторов, счетчик идентификаторов, установленный в единицу.

Листинг 1. Функция **__init__**

```
1 def __init__(self):
2     self.errors = []
3     self.lex_table = []
4     self.identifiers = {}
5     self.id_counter = 1
```

Функция **reset** предназначена для очистки списков и таблиц анализатора для его повторного использования.

Вход: заполненные служебные таблицы и списки

Выход: пустые служебные таблицы и списки, счетчик идентификаторов, установленный в единицу.

```

1 def reset(self):
2 self.errors.clear()
3 self.lex_table.clear()
4 self.identifiers.clear()
5 self.id_counter = 1

```

2.2.1 Основные методы

Метод `del_comments` предназначен для удаления комментариев и нормализации пробелов. Сначала все переводы строки заменяются на пробелы, а затем множественные пробелы сжимаются до единичных. Далее с помощью регулярного выражения выделяются и удаляются все закрытые комментарии, а при поиске незакрытых, помимо удаления, в таблицу ошибок добавляется строка о незакрытом комментарии.

Вход: изначальный текст.

Выход: текст без переносов, множественных пробелов и комментариев.

Листинг 3. Функция `del_comments`

```

1 def del_comments(self, expression: str) -> str:
2 self.errors = []
3 print("Initial expression:", expression)
4 expression = re.sub(r'[\n\r]+', ' ', expression)
5 expression = re.sub(r'\#.*?\#', '', expression)
6 if (re.search(r'\#', expression)):
7 self.errors.append('Ошибка! Незакрытый комментарий')
8 expression = re.sub(r'\#.*$', '', expression)
9 expression = re.sub(r'\s+', ' ', expression)
10 expression = expression.strip()
11 return expression

```

Метод `tokenize` предназначен для разбиения входной строки на лексемы. В нем задается таблица правил, которая определяет, какая последовательность символов является той или иной лексемой. Пока строка не становится пустой, последовательно проверяются шаблоны из таблицы. Если найдено совпадение, то токен добавляется в таблицу с помощью вспомогательной функции `add_token`, а затем найденный токен удаляется из изначальной строки. Для римских цифр и идентификаторов используются вспомогательные функции. Если последовательность не подошла ни одному шаблону, фиксируется ошибка и символ также удаляется.

Вход: текст без комментариев.

Выход: заполненная таблица лексем.

Листинг 4. Функция `tokenize` и вспомогательная `add_token`

```

1 def add_token(self, token_type, value):
2 self.lex_table.append((token_type, value))
3
4 def tokenize(self, expression: str):
5 self.lex_table = []
6 token_patterns = [

```

```

7 ("Знак присваивания", r'^:='),
8 ("Разделитель", r'^;'),
9 ("Плюс", r'^\+'),
10 ("Минус", r'^-'),
11 ("Умножение", r'^\*'),
12 ("Деление", r'^/'),
13 ("Левая скобка", r'^\('),
14 ("Правая скобка", r'^\)'),
15 ("Идентификатор", reg_id_big),
16 ("Римское число", reg_roman_all),
17 ("Пробел", r'^\s+'),
18 ]
19
20 while expression:
21     expression = expression.lstrip()
22     matched = False
23     for token_type, pattern in token_patterns:
24         match = re.match(pattern, expression)
25         if match:
26             value = match.group()
27             if value == " ":
28                 continue
29             elif token_type == "Пробел":
30                 expression = expression[len(value):]
31                 matched = True
32                 break
33             elif token_type == "Римское число":
34                 or_value = self.cor_roman(value)
35                 expression = expression[or_value:]
36                 matched = True
37                 break
38             elif token_type == "Идентификатор":
39                 or_value = self.cor_id(value)
40                 expression = expression[or_value:]
41                 matched = True
42                 break
43             else:
44                 self.add_token(token_type, value)
45                 expression = expression[len(value):]
46                 matched = True
47                 break
48             break
49         if not matched:
50             self.errors.append(f"Ошибка! Недопустимый символ: {expression[0]}")
51             expression = expression[1:]
52
53

```

Метод `cor_id` предназначен для выявления ошибок внутри идентификатора и их разрешения. Если длина идентификатора больше 15 символов, то добавляется ошибка, идентификатор в список лексем не добавляется. При верном размере проверяется, нет ли в идентификаторе недопустимых символов или не начинается ли он с числа. В случае успешных проверок идентификатору присваивается уникальный номер и он добавляется в список идентификаторов. Если такой идентификатор уже был найден раньше, для него используется тот уникальный номер, который находится в списке идентификаторов.

Вход: строка, представляющая идентификатор.

Выход: длина исходной строки.

Листинг 5. Функция `cor_id`

```
1 def cor_id(self, value: str):
2 or_value = value
3 if len(value) > 15:
4 self.errors.append(f"Ошибка! Идентификатор '{value} превышает' 15 символов. "
5 "Необходимо разбить на идентификаторы меньшей длины")
6 else:
7 if not re.fullmatch(reg_id_all, value):
8 if re.fullmatch(reg_roman_all, value):
9 self.cor_roman(value)
10 else:
11 digits = re.search(r'[0-9]+', value)
12 roman = re.search(r'^[XVI]+', value)
13 if digits:
14 self.errors.append(f"Ошибка! Идентификатор '{value}' содержит некорректные
15 символы '{digits.group()}'")
16 if roman:
17 self.errors.append(f"Ошибка! Идентификатор '{value}' содержит некорректные
18 символы '{roman.group()}' в начале")
19 else:
20 if value not in self.identifiers:
21 self.identifiers[value] = self.id_counter
22 self.id_counter += 1
23 self.add_token("Идентификатор", value)
24 return len(or_value)
```

Метод `cor_roman` предназначен для приведения римских чисел в «канонический» вид (с соблюдением правил записи римских чисел). Если вся строка не соответствует правилам, добавляем ошибку в список и ищем подстроки, соответствующие корректной записи чисел, при нахождении добавляем число в список лексем.

Вход: строка, представляющая римское число.

Выход: длина исходной строки.

Листинг 6. Функция `cor_roman`

```
1 def cor_roman(self, value: str):
2 valid = re.fullmatch(reg_roman, value)
3 or_value = value
4 if not valid:
5 self.errors.append(f"Ошибка! Римское число записано не по правилам: {value}")
6 while value:
7 part = re.match(reg_roman, value)
8 if part and part.group():
9 token_val = part.group()
10 self.add_token("Римское число", token_val)
11 value = value[len(token_val):]
12 else:
13 break
14 else:
15 self.add_token("Римское число", value)
16 return len(or_value)
```

2.2.2 Регулярные выражения

Регулярные выражения используются для проверки корректности лексем. В отчете рассмотрены четыре наиболее сложных регулярных выражения.

Регулярное выражение для римских чисел
`reg_roman = r'^(X{0,3})(IX|IV|V?I{0,3})'`. Структура:

- `^(X{0,3})` необязательная часть - может быть от 0 до 3 символов X
- `(IX|IV|V?I{0,3})` - группа, которая состоит из трех комбинаций, из которых может быть только одна: либо IX, либо IV, либо повторяющаяся 0 или 1 раз V и повторяющаяся от 0 до 3 раз I.

Таким образом, регулярное выражение включает все возможные корректные комбинации римских цифр от 1 до 39, состоящих из X, V, I.

Регулярное выражение для римских чисел, не учитывающее правила построения `reg_roman_all = r'^[XVI]+'`. Структура: `^[XVI]+` последовательность из X, V, I от одного символа и больше с начала строки.

Регулярное выражение для идентификаторов, которые состоят только из латинских букв (не могут начинаться с X, V, I больших) и ограничены по длине 15 символами

`reg_id = r'^(?![XVI]+)[a-zA-Z][a-zA-Z]{0,14}'`. Структура:

- `^(?![XVI]+)` первый символ (или последовательность) не состоит из символов X, V, I.
- `[a-zA-Z]` первый символ - любая буква латинского алфавита.
- `[a-zA-Z]{0,14}` - последующие символы с такими же ограничениями (от 0 до 14 символов)

Регулярное выражение для идентификаторов из всех латинских букв и без учета длины `reg_id_all = r'^[a-zA-Z]+'`. Структура: `[a-zA-Z]+` последовательность из любых латинских букв от одного символа и больше в начале строки.

3 Результаты работы программы

Первый пример входного текста:

```
XXXXXVII abcdefghijklmnopqrstuvwxyz := : +-* / xXx;  
#comment#  
x := XX k.
```

В данной программе римское число записано не по правилам, есть недопустимый символ `:` и слишком длинный идентификатор. Далее приведен вывод лексического анализа входного текста:

```
Ошибка! Римское число записано не по правилам: XXXXXVII  
Ошибка! Идентификатор 'abcdefghijklmnopqrstuvwxyz' превышает 15 символов.  
Ошибка! Недопустимый символ: :
```

Таблица лексем:

Лексема	Тип лексемы	Значение
XXX	Римское число	30
XXVII	Римское число	27
:=	Знак присваивания	
+	Плюс	
-	Минус	
*	Умножение	
/	Деление	
xXx	Идентификатор	xXx : 1
;	Разделитель	
x	Идентификатор	x : 2
:=	Знак присваивания	
XX	Римское число	20
k	Идентификатор	k : 3

Второй пример входного текста:

```
abcbcbcb := +(X*VIII)/(XXV-III);  
#lockedcomment wow that's good#  
abcbcbcb ) ( XXV ; +.
```

В данной программе нет ошибок, выводится только таблица лексем. Далее приведен вывод лексического анализа входного текста:

Таблица лексем:

Лексема	Тип лексемы	Значение
---------	-------------	----------

abcbcbcb	Идентификатор	abcbcbcb : 1
:=	Знак присваивания	
+	Плюс	
(	Левая скобка	
X	Римское число	10
*	Умножение	
VIII	Римское число	8
)	Правая скобка	
/	Деление	
(	Левая скобка	
XXV	Римское число	25
-	Минус	
III	Римское число	3
)	Правая скобка	
;	Разделитель	
abcbcbcb	Идентификатор	abcbcbcb : 1
)	Правая скобка	
(	Левая скобка	
XXV	Римское число	25
;	Разделитель	
+	Плюс	

Третий пример входного текста:

```
ababab12 XX + XVIIIII; ababab Ia::= #unlocked comme
nt how dare
ff ; .
```

В данной программе незакрытый комментарий, идентификатор с символами не из входного языка, идентификатор, начинающийся с числа, недопустимые символы :, = и неверное римское число. Далее приведен вывод лексического анализа входного текста:

```
Ошибка! Незакрытый комментарий
Ошибка! Идентификатор 'ababab12' содержит некорректные символы '12'
Ошибка! Римское число записано не по правилам: XVIIIII
Ошибка! Идентификатор 'Ia' содержит число 'I' в начале
Ошибка! Недопустимый символ: :
Ошибка! Недопустимый символ: =
```

Таблица лексем:

Лексема	Тип лексемы	Значение

XX	Римское число	20
+	Плюс	
XVIII	Римское число	18
II	Римское число	2
;	Разделитель	
ababab	Идентификатор	ababab : 1
:=	Знак присваивания	

Заключение

В ходе выполнения лабораторной работы был разработан лексический анализатор для языка, который содержит только латиницу, арифметические выражения, разделенные символом ; (точка с запятой). Арифметические выражения состоят из идентификаторов (максимум 15 символов), римских чисел (состоящих из больших букв X, V, I), знака присваивания ($:=$), знаков операций $+$, $-$, $*$, $/$ и круглых скобок. Данный язык является автоматным, так как представим в виде синтаксической диаграммы.

Реализованная программа также удаляет комментарии, проводит лексический анализ текста, формирует и выводит таблицу лексем с указанием их типов, обнаруживает и выводит таблицу ошибок.

Достоинства программы:

- данные читаются последовательно, что позволяет обрабатывать большие файлы;
- использование регулярных выражений и таблицы правил соответствия позволяет масштабировать программу.

Недостатки программы:

- при обнаружении ошибки не сохраняется ее позиция;
- отсутствие разделителя означает единое арифметическое выражение.

Масштабирование программы:

- использование логирования вместо выводов ошибок через функцию `print`;
- расширение языка посредством увеличения римских цифр.

Список использованной литературы

- [1] Секция «Телематика». — Текст : электронный // tema.spbstu.ru : [сайт]. URL: <https://tema.spbstu.ru/algorithm/> (дата обращения: 25.02.2026).
- [2] Сети, Р.; Ахо, А. Компиляторы: принципы, технологии и инструментов - М.: Издательство «Наука», 2008. - 1178 с. (дата обращения: 25.02.2026).
- [3] Ю.Г. Карпов Теория автоматов. - СПб: Издательский дом «Питер», 2003. - 208 с. (дата обращения - 24.02.2026).